

0.87.1. Agrandir une lettre : texteGrand — toutes les tailles

Le même A, aux tailles 0, 1, 2 et 3, côte à côte : un seul nombre change d'une ligne à l'autre.

1. Ce qu'il faut comprendre

Quatre lignes, un seul nombre qui change : la taille, 0, 1, 2, puis 3. On voit le A grandir.

Tu choisis la taille, la console s'adapte : aucune taille n'est dessinée à l'avance. Chaque `texteGrand` fait calculer les tuiles de SA taille.

La place : taille 0, 1 case ; taille 1, 2 × 2 ; taille 2, 3 × 3 ; taille 3, 4 × 4. Toujours **taille + 1** cases de côté. Les colonnes de départ (0, 2, 5, 9) laissent juste la place à chacun.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1753 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          73 o ← la console, à l'allumage - la mise en route de la console
texte_grand_0()     48 o ← main() - ta fonction
texte_grand_1()     97 o ← main() - ta fonction
texte_grand_2()    146 o ← main() - ta fonction
texte_grand_3()    195 o ← main() - ta fonction
main()              46 o ← démarrage - ta fonction
VBlank              8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage       28 o ← image() - attend l'image suivante
AttendreVBlank      16 o ← démarrage - attend que l'écran soit libre
AdresseCase         10 o ← poser() - trouve l'adresse d'une case de l'écran
CopierTuiles        18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage - vide les deux cartes de l'écran
Tuiles              1024 o ← CopierTuiles - la police entière, et 20 tuiles dessinées
GRAND_TABLE_0_0_0    1 o ← poser() - une table gravée
GRAND_TABLE_1_0_1    2 o ← poser() - une table gravée
GRAND_TABLE_1_1_2    2 o ← poser() - une table gravée
GRAND_TABLE_2_0_3    3 o ← poser() - une table gravée
GRAND_TABLE_2_1_4    3 o ← poser() - une table gravée
GRAND_TABLE_2_2_5    3 o ← poser() - une table gravée
GRAND_TABLE_3_0_6    4 o ← poser() - une table gravée
GRAND_TABLE_3_1_7    4 o ← poser() - une table gravée
GRAND_TABLE_3_2_8    4 o ← poser() - une table gravée
GRAND_TABLE_3_3_9    4 o ← poser() - une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond,
```

EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()          × 10   302 o sur place, dans texte_grand_0(), texte_grand_1(), texte_grand_2(), texte_grand_3() +
AdresseCase (10 o), GRAND_TABLE_0_0_0 (1 o), GRAND_TABLE_1_0_1 (2 o), GRAND_TABLE_1_1_2 (2 o), GRAND_TABLE_2_0_3 (3
o), GRAND_TABLE_2_1_4 (3 o), GRAND_TABLE_2_2_5 (3 o), GRAND_TABLE_3_0_6 (4 o), GRAND_TABLE_3_1_7 (4 o),
GRAND_TABLE_3_2_8 (4 o), GRAND_TABLE_3_3_9 (4 o)
image()          × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
texte_grand_0() × 1    3 o sur place, dans main()
texte_grand_1() × 1    3 o sur place, dans main()
texte_grand_2() × 1    3 o sur place, dans main()
texte_grand_3() × 1    3 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texteGrand> // écrit un texte agrandi, taille de 0 à 10
```

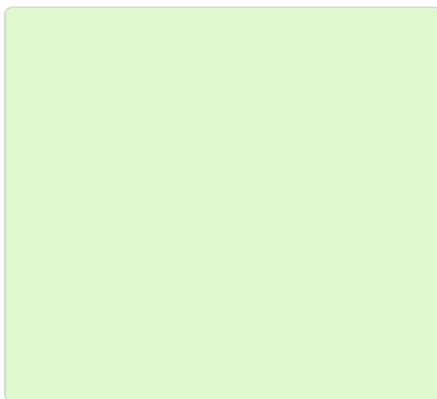
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texteGrand(0, 0, "A", 0); // taille 0 : 1 case, la lettre normale
    texteGrand(2, 0, "A", 1); // taille 1 : 2 × 2 cases
    texteGrand(5, 0, "A", 2); // taille 2 : 3 × 3 cases
    texteGrand(9, 0, "A", 3); // taille 3 : 4 × 4 cases

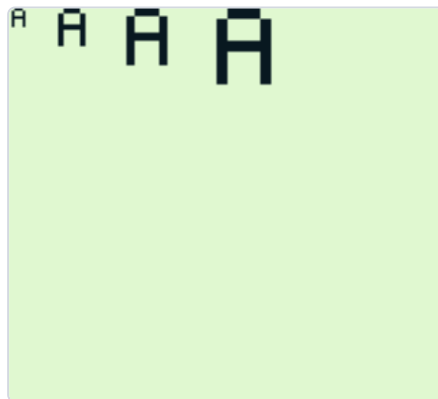
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Quatre A côte à côte, de plus en plus grands : tailles 0, 1, 2 et 3.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ taille 0 : une case
- ✓ taille 3 : dans ses 4×4 cases
- ✓ chacun plus grand que le précédent

Ensuite : **0.87.2. Agrandir une lettre : `texteGrand` — dix fois** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche
Game Boy en C++